

ResQGrid AI — Documentary and Demonstration Script

From a report to a human-approved response | Narration, scenes, recording plan, and presentation notes

Production edition 1.0 | Target running time: approximately 15 minutes | Source baseline: 0d85c00

This is a recording-ready script, not a rendered video. All incident footage must use fictional demonstration data. Keep the notice “Demonstration prototype — not for real emergency use” visible at the opening and closing, and identify mock analysis whenever shown.

SOURCE-BASED EDITION · FICTIONAL DEMONSTRATION DATA ONLY

Contents

1. Production brief 3

2. Master timeline 4

3. Recording preparation 5

4. Fictional scenario and expected checkpoints 6

5. Scene 1 — The coordination problem 7

6. Scene 2 — What the project is 8

7. Scene 3 — A citizen submits a report 9

8. Scene 4 — Queue and map context 10

9. Scene 5 — Triage without a black box claim 11

10. Scene 6 — Why this priority score? 12

11. Scene 7 — Choosing a resource 13

12. Scene 8 — The human approval boundary 14

13. Scene 9 — Responder progress 15

14. Scene 10 — Evidence and command assistant 16

15. Scene 11 — How the system is built 17

16. Scene 12 — Quality, security, and limitations 18

17. Scene 13 — Roadmap and conclusion 19

18. Optional inserts and extended technical cut 20

19. Recording run sheet 21

20. Failure and fallback plan 22

21. Suggested interview and evaluator questions 23

22. Ninety-second cut-down script 24

23. Editorial quality checklist and sources 25

1. Production brief

The documentary explains what ResQGrid AI is, why the project was built, how the main workflow operates, which features are implemented, how the system is engineered, and what it cannot yet guarantee. It is intended for a project presentation, academic demonstration, portfolio walkthrough, or hackathon evaluation.

The central story follows one fictional flood report from a citizen through dispatcher review and resource assignment to responder completion. The film combines a calm voice-over, real application screen recordings, simple architecture graphics, and short explanatory cards. Avoid real disaster footage or distressing imagery unless separately licensed and ethically justified; it is not necessary to explain this prototype.

Use a clear, factual tone. The narration must not claim lives saved, response-time reductions, certified accuracy, government integration, autonomous dispatch, guaranteed offline delivery, or a production SLA. This project is proprietary, not automatically open source because its repository is accessible. Keep human decision-making central.

The 15-minute timeline is an editorial target. Read at approximately 125–145 words per minute and use the remaining time for cursor movements, screen holds, role changes, diagrams, and transitions. Rehearse against the actual application latency; adjust pauses rather than fabricating a faster result. The written narration does not require continuous speech throughout every scene.

Recommended deliverable for the editor: 1920×1080 landscape video at a consistent 30 fps, clean speech audio, optional restrained instrumental background music, readable captions, and restrained zooms on small UI text. These are production recommendations, not project runtime requirements.

2. Master timeline

Scene	Time	Subject	Principal visual
1	00:00–00:50	The coordination problem	Title, fictional report cards, safety notice.
2	00:50–01:50	Product idea and roles	Dashboard overview and four-role graphic.
3	01:50–03:10	Citizen report	Report modal, pin, and successful submission.
4	03:10–04:10	Shared operational view	Queue, map selection, details, polling label.
5	04:10–05:35	Local and optional AI triage	Triage action, output, ensemble graphic.
6	05:35–06:50	Explainable priority	Numeric factors and worked-example card.
7	06:50–08:15	Resource matching	Candidate cards, hazards, estimated ETA caveat.
8	08:15–09:25	Human approval	Approve once, assignment appears, resource reserved.
9	09:25–10:35	Responder lifecycle	Accept, en route, on scene, complete.
10	10:35–11:45	Evidence and assistant	Protected image preview and contextual question.
11	11:45–13:00	Architecture, offline support, optimizer	Architecture card, optional API-only demonstration.
12	13:00–14:10	Quality, security, and limitations	Test evidence and honest capability boundaries.
13	14:10–15:00	Roadmap and conclusion	Future-work card, principle, safety/license credits.

3. Recording preparation

Use an isolated local demonstration environment whenever possible. Do not reset, populate, or repeatedly mutate a shared hosted database merely to obtain footage. Confirm the selected workspace/deployment and prepare an operator-approved disposable dataset. The full setup guide is `docs/PROJECT_DOCUMENTATION.md`.

Preflight checklist:

- Start PostgreSQL/Redis and the API/web application; verify a database-backed request, not only the static health endpoint.
- Verify the map renders streets, labels, and incident markers. Confirm both MapLibre worker assets are available if using the fallback engine.
- Prepare citizen, dispatcher, and responder sessions in separate browser profiles/devices. Tabs in the same profile share localStorage tokens.
- Confirm an appropriate resource is available and note which fictional incident will be used. Avoid hard-coding an expected resource name in the narration.
- Decide whether the recording uses local-only text triage or a configured external model. Verify the mode before labeling the footage.
- If using no provider keys, prepare an explicit “Mock image analysis” overlay. Never describe that canned output as real vision recognition.
- Prepare a harmless licensed or self-created sample image. Do not include identifiable people, private metadata, medical records, or genuine distress scenes.
- Hide password managers, private browser tabs, API keys, bearer tokens, environment files, and personal account information from the capture.
- Rehearse the assignment state sequence and verify resource release. Remember that incident closure is separate.
- Record a clean test-result frame only after actually running the tests. Historical results should be labeled as such.
- Test microphone level, captions, screen scaling, and cursor visibility. Ensure map/provider attribution remains visible.

Do not record secret configuration screens as a shortcut to explaining architecture. Use variable names and placeholders on a prepared graphic. The publicly seeded demo credentials can be used by the operator, but they need not be read aloud or enlarged on screen.

4. Fictional scenario and expected checkpoints

Use this scenario text for consistent input:

Title: DEMO – Flooded access road
Type: flood
Description: Fictional exercise. Rising water has stranded 20 people near an access road. Some people may need medical assistance.
Location: Review the demo pin near 6.9271, 79.8612
People at risk: 20
Medical need: yes

The scenario is not a report about the real location. Prefixing the title with DEMO is important. The current report modal does not expose every API field, so do not claim the UI entered four vulnerable people or a precise injury count unless an additional authorized API step actually did so.

Checkpoints to capture: successful creation; selected record in the dispatcher queue; triage output; calculated priority; candidate recommendation; human approval; an assignment card; responder status progression; completed assignment and released resource. Score, confidence, ordering, and candidate names may vary with age, existing data, provider output, and resource state.

The 75.5 priority example later in this script is a separate illustrative calculation with explicit assumed inputs. Do not substitute that graphic for the application's actual result or imply that this particular report must score 75.5.

5. Scene 1 — The coordination problem

Time: 00:00–00:50. **Visual:** Fade from a dark background into the project title. Show fictional text cards labeled Report, Location, Resources, and Decisions converging into a dashboard outline. Display the safety notice immediately.

Narration:

“When information arrives from different places, the challenge is not only collecting it. Someone must understand the report, locate the problem, compare available resources, and decide what to do next. ResQGrid AI explores how those steps can work together in one coordinated interface. It is an educational decision-support prototype built around a simple principle: AI recommends, and humans approve. In this film, every incident is fictional. We will follow one report through the system and explain both the technology and its limits.”

On-screen text: “ResQGrid AI — Intelligent Emergency Resource Network.” Secondary line: “Demonstration prototype — not for real emergency use.”

Editing note: Use a measured opening, not sirens or dramatic claims about guaranteed outcomes. Hold the safety notice long enough to read.

6. Scene 2 — What the project is

Time: 00:50–01:50. **Visual:** Show the desktop dashboard without clicking actions. Slowly highlight KPI bar, incident queue, map, and detail panel. Add a simple Citizen / Dispatcher / Responder / Administrator role graphic.

Narration:

“ResQGrid AI is a web-based command dashboard. Citizens can submit reports. Dispatchers review information, run advisory triage, compare priorities, and authorize resource assignments. Responders record the progress of their assigned work, while administrators share the main operational privileges. The map adds geographic context, and the queue helps organize the reports that the current user can access. This is not an official emergency call center or an autonomous rescue service. It demonstrates how software can make a coordination workflow more visible and explainable while keeping a person responsible for the decision.”

Action: Show the role badge. Do not suggest that every role sees every incident or has identical controls.

On-screen text: “Report → Review → Prioritize → Recommend → Human approval → Field progress.”

7. Scene 3 — A citizen submits a report

Time: 01:50–03:10. **Visual:** In the citizen profile, open Report New Incident. Enter the prepared fictional scenario. Show map selection and coordinate confirmation, then people count and medical need. Submit once and wait for the actual result.

Narration:

“Our example begins with a fictional flood. The reporter chooses an incident category, writes a description, and identifies the location. A map pin, place search, or permitted device location can help, but the reporter must still check the coordinates. The initial map center is not proof of where an incident happened. The form also accepts a people-at-risk count and a medical-need flag. After submission, the server records the report and its owner. At this point it is a report awaiting assessment, not a verified event and not an automatic dispatch.”

Action: Hold the completed form for several seconds before submitting. After success, show the new title and selected record. Use a harmless location and explicitly fictional description.

On-screen text: “Reporter input ≠ verified fact.” Then: “Submitted successfully” only if the actual request succeeded.

Fallback: If the report is queued offline, do not continue as if it reached the dispatcher. Use the offline-support insert in chapter 18 and verify server delivery before resuming.

8. Scene 4 — Queue and map context

Time: 03:10–04:10. **Visual:** Switch to the dispatcher profile. Wait for the report to appear or refresh. Click the incident card, then its map marker. Briefly show severity filters and return to the target incident.

Narration:

“The dispatcher sees accessible reports in the incident queue and on the map. Selecting either opens the same incident detail. The queue sorts loaded records by priority score, then severity, then creation time. The dashboard refreshes core information through HTTP polling, normally every fifteen seconds, and after local actions. It is not a WebSocket feed, and delays can occur. Map markers show stored locations; they are not continuous live vehicle tracking. The operator combines this spatial view with the original report instead of treating a colored marker as the whole situation.”

On-screen text: “Core dashboard: 15-second polling.” Small note: “Loaded records, global counters, and assistant context can have different scopes.”

Editing note: Avoid a cut implying that the report reached another device instantly. Preserve a short real wait or label a time-compressed transition.

9. Scene 5 — Triage without a black box claim

Time: 04:10–05:35. **Visual:** Click Run AI Triage. Show returned severity, confidence, reason codes, and needs. Overlay a two-input diagram: local engine plus optional configured provider feeding a validated result.

Narration:

“Triage begins with a local Python text engine. It uses English phrases, limited negation handling, severity signals, and simple count extraction. When a cloud model is configured, the system also requests a structured interpretation and combines the two outputs. The current provider selection prefers Gemini when its key is present; otherwise it can use Alibaba Model Studio. It does not automatically cycle through every provider after a failure. Without a cloud model, local text triage remains available through the server. The result includes severity, confidence, immediate needs, and reason codes. These are advisory signals. Confidence is not certified accuracy, and ambiguous language or incomplete reports can still produce errors.”

Action: Read one real reason code from the visible output. Do not invent a specific code or exact confidence before recording. If the provider fails and the local result is used, label that actual mode.

On-screen text: “Local text engine + optional LLM.” Then: “Human verification required.”

Optional insert: Show sanitized incident JSON with the ensemble block. Label it “API data”; there is no dedicated ensemble-disagreement panel in the current UI.

10. Scene 6 — Why this priority score?

Time: 05:35–06:50. **Visual:** Click Calculate Priority. Zoom into the numeric factor values and score. Cut to a prepared six-factor graphic, then return to the actual result.

Narration:

“The priority score is a deterministic calculation, separate from the language model. Its default factors are life risk, medical urgency, people at risk, environmental risk, time sensitivity, and evidence confidence. Each has an explicit weight. The operator can see the numeric components and understand the result. For illustration, a high-severity flood with specified counts, medical need, twenty minutes of age, and eighty-percent triage confidence produces seventy-five point five under the documented example inputs. That is an example, not a fixed score for every flood. Time changes the calculation, and the stored score must be recalculated deliberately. These weights are configurable heuristics, not clinically validated emergency policy.”

Graphic: “Illustrative inputs only: life 75; medical 100; people 60; environment 80; time 40; confidence 80 → weighted score 75.5.” State that the people factor assumes 20 people and four vulnerable people, with no reported injuries.

On-screen text: “Explainable calculation ≠ scientifically validated outcome.”

11. Scene 7 — Choosing a resource

Time: 06:50–08:15. **Visual:** Click Get Recommendations. Show at least two candidates if available, their types, confidence, ETA, and reasons. Add a simple straight-line diagram beside a hazard circle, explicitly labeled illustrative.

Narration:

“Next, the system suggests available resources. For a flood, preferred categories include rescue boats, helicopters, and rescue teams. The matcher checks type, availability, range, approximate distance, and a penalty for nearby active hazards. The travel estimate uses a simple distance-and-speed formula. It is not road-network routing, live traffic, or a guarantee of the safest path. Capacity, equipment, operating hours, and actual access conditions still need operator review. A high ranking does not automatically make a resource suitable. These recommendation cards organize the decision; they do not replace it.”

Action: Compare visible candidates without claiming the nearest one is always best. If only one is available, say so; do not fabricate an alternative in the footage.

On-screen text: “Estimated ETA — not live routing.” Secondary: “Review actual suitability and conditions.”

Optional narration if space permits: “Repeated recommendation generation can create additional pending records, so the demonstration requests a set once and examines it carefully.”

12. Scene 8 — The human approval boundary

Time: 08:15–09:25. **Visual:** Pause with the pending recommendation visible. Highlight Approve and Reject. Approve one suitable fictional candidate once. Capture the dispatched-unit card and resource state afterward.

Narration:

“This is the key decision boundary. A dispatcher reviews the proposal and chooses whether to approve or reject it. The dashboard's approval action first approves the recommendation and then requests an assignment. The backend checks that the resource is still available before reserving it. Those are separate requests, so an approval alone does not prove that dispatch succeeded. If a conflict occurs, the operator checks current state instead of repeatedly clicking. In this successful demonstration, an assignment now exists. The software has recorded an authorized reservation; it has not contacted an official emergency service or physically sent a vehicle.”

On-screen text: “Human action required.” Then: “Approval → assignment request → resource reservation.”

Action: If assignment creation fails, record the limitation honestly or use a separately rehearsed successful take. Do not splice the success state onto a failed request without explaining the change.

13. Scene 9 — Responder progress

Time: 09:25–10:35. **Visual:** Switch to the responder profile and select the incident. Advance the fictional task through the permitted sequence. Use labeled time-compressed transitions between simulated travel and arrival.

Narration:

“The responder can now accept eligible work. If the dispatch has no assigned responder, the first successful acceptance claims it. The server protects that claim with a row lock, and another responder cannot simply take over the task. Progress follows assigned, accepted, en route, on scene, and completed. A responder cannot skip arbitrary steps or cancel a dispatch; cancellation belongs to a dispatcher or administrator. On completion, the resource returns to the available pool when its assignment reference matches. The incident itself does not automatically close. Assignment progress and incident resolution are separate records in this prototype.”

On-screen text: “Simulated field progression — time compressed.” Show “Incident closure is separate” at the end.

Editing note: Do not claim SMS, push, or radio notification. Work visibility is through the application and human coordination; no such notification integration was found.

14. Scene 10 — Evidence and command assistant

Time: 10:35–11:45. **Visual:** Return to dispatcher. Show a previously uploaded harmless sample image and its actual analysis mode. Then open the assistant and ask “Give me a situation summary.”

Narration:

“Evidence can add context to a report. Uploaded raster images are validated and stored as sanitized PNG copies, and users retrieve them through an authorized endpoint. A configured multimodal provider can suggest visible signals, but its output remains uncertain and does not automatically change the incident's priority. Without provider keys, image analysis is a canned mock response and must be labeled that way. The command assistant offers another view: it uses a bounded database context to answer situational questions. Its summaries can be cached and may cover only a subset of incidents, so important statements still need verification against the records.”

On-screen labels: “Sanitized image copy”; “Advisory image analysis” or “Mock image analysis,” according to actual mode; “Assistant: bounded context, not every record.”

Action: If upload analysis fails, show that the image can remain stored while analysis is unavailable. Avoid reading private file names or account details aloud.

15. Scene 11 — How the system is built

Time: 11:45–13:00. **Visual:** Show a clean architecture diagram. Highlight browser/PWA/Android wrapper, Next.js, FastAPI, PostgreSQL, Redis, optional model provider, and local/OSS storage. Use short inserts for outbox and API optimizer.

Narration:

“The interface is built with Next.js, React, and TypeScript. A FastAPI application handles authentication, records, analysis, recommendations, and assignments. PostgreSQL stores the data, while Redis shares selected request-limit counters. Evidence uses local storage or private object storage, depending on configuration. The same web interface can be packaged for Android with Capacitor. Offline support is limited to shell caching and attempted report queuing on the device; it does not make the complete system work without a server. A separate API endpoint also offers Hungarian assignment optimization across multiple incidents and resources. That endpoint returns an advisory plan, not an automatic dispatch, and it has no dedicated dashboard control today.”

On-screen text: “One modular backend, optional external integrations.” Small note: “API-only optimizer; heuristic cost comparison.”

Editing note: If showing optimizer output, use a sanitized real response or clearly labeled illustrative graphic. Its savings field is a heuristic comparison, not measured real-world time saved.

16. Scene 12 — Quality, security, and limitations

Time: 13:00–14:10. **Visual:** Show readable test evidence, then a security checklist and a concise limitations card. Avoid fast-scrolling source code or secrets.

Narration:

“During preparation of the project guide, the backend suite passed one hundred tests and the frontend regression suite passed thirteen. Those checks cover important behavior, but they do not prove real-world emergency readiness. The implementation includes role and object authorization, separate access and refresh tokens, selected rate limits, private evidence delivery, and security headers. Public privileged demo accounts are intentionally retained, which is why this environment must contain only fictional data. Other limits include partial offline support, approximate routing estimates, selective audit coverage, cached summaries, and incomplete operational tooling. Honest documentation makes those boundaries visible rather than treating a successful demonstration as certification.”

On-screen text: “Test results are revision-specific.” Then: “Not certified. No real emergency or personal medical data.”

Editing note: If the tests are rerun for a later revision, replace the numbers with actual results. If displaying the documentation-time results, label the source baseline 0d85c00.

17. Scene 13 — Roadmap and conclusion

Time: 14:10–15:00. **Visual:** Display three future-work columns: Reliability, Decision Support, Operations. Return to a dashboard-wide shot, then the closing principle and safety/license credits.

Narration:

“The next steps are stronger workflow recovery, safer offline delivery, more complete operator screens, better evaluation of triage, real routing integrations, and tested deployment recovery. These are future improvements, not features already delivered. ResQGrid AI demonstrates a connected path from a report to an explainable recommendation and a human-approved assignment. Its value as a project is in making that path visible, testable, and open to improvement. The central principle remains the same: AI assists the judgment; people remain responsible for the decision.”

Closing card: “AI recommends. Humans approve.” Below it: “Demonstration and research prototype — not for real emergency use.”

Credits: “ResQGrid AI Team. Copyright 2026. All Rights Reserved. Software and documentation are proprietary; see LICENSE and DISCLAIMER.” Include relevant map/provider and separately licensed media credits. Do not imply official endorsement.

18. Optional inserts and extended technical cut

These clips can replace pauses or extend the film to approximately 18–20 minutes. Label them clearly and adjust the master timeline if included. Do not insert all of them while retaining a false 15-minute duration claim.

Offline report insert, 45–60 seconds: Start from an already loaded, authenticated local demo. Simulate loss of network access in a controlled recording environment. Submit one fictional report and show the queued message. Reconnect with the same account, reopen/trigger replay as needed, and verify the record server-side. Narration: “This payload is queued on the device. It is not yet received by the dispatcher. Delivery still depends on local storage, a valid session, and successful replay.” Explain that no evidence image is queued with it.

Optimizer insert, 45–60 seconds: In a sanitized API client, call `/assignments/optimize` as an authorized dispatcher with a deliberate eligible incident-ID set. Hide the bearer token. Show pairings, unmatched incidents, and algorithm name. Narration: “This plan minimizes the current heuristic pair cost. It does not include every operational constraint or incident priority, and it does not reserve resources.” Do not translate `savings_km` into a proven real-world benefit.

API contract insert, 30–45 seconds: Show `/docs`, the incident schema, and the three triage/priority/recommendation actions. Keep Authorize contents hidden. Explain request validation, lowercase status values, and object access. Avoid submitting an extra live mutation merely to fill screen time.

Build and deployment insert, 45–60 seconds: Show a prepared variable-name card and repository folders, then a recorded successful local build if available. Explain Vercel web hosting, Render API hosting, PostgreSQL, optional Redis/OSS, and exact CORS/API origins. Note that public frontend variables are compiled into the bundle. Never film actual private environment values.

Map engineering insert, 30–45 seconds: Show the MapLibre worker/shared file names and a successful network response in a secret-free view. Explain that both modules are published together and the worker URL is configured before map initialization. Do not show a broken map as a completed final feature.

19. Recording run sheet

Order	Operator action	Success evidence to retain
1	Start isolated environment and verify sessions/map	Dashboard and clean map footage.
2	Create one DEMO report as citizen	Created incident ID and selected title.
3	Open same report as dispatcher	Correct report, coordinates, and role.
4	Run triage and inspect actual mode	Real output and sanitized mode details if needed.
5	Calculate priority	Actual score and numeric component values.
6	Generate recommendations once	Candidate data and pending status.
7	Approve one candidate	Recommendation state plus actual assignment card.
8	Claim as responder	Accepted state and ownership.
9	Advance simulated field steps	En-route, on-scene, completed states.
10	Verify release and separate incident state	Resource available; incident status clearly shown.
11	Show evidence and assistant	Actual analysis mode and bounded summary.
12	Record architecture/tests/limitations cards	Revision-specific evidence, no unsupported claims.
13	Review final sequence	Consistent incident identity, safe data, readable captions.

Keep an editorial log linking each clip to the fictional incident and source revision. If multiple takes are used, preserve continuity and label time compression. Do not imply that different records are the same record or that generated diagrams are live application screenshots.

20. Failure and fallback plan

If the API is unavailable, use previously recorded real footage labeled with its recording context or show a clearly marked architectural explanation. Do not use static images to imply a live successful operation. The health endpoint alone is insufficient proof that login/database workflows function.

If the map fails, check worker assets, tiles, network connectivity, and WebGL before recording. Do not hide attribution or weaken security controls. If the provider is unavailable, demonstrate actual local text triage/database assistant behavior and label image analysis mock or unavailable.

If no resource is available, explain the state or prepare a fresh isolated dataset before the take. Do not reset shared data or fabricate a candidate. If approval succeeds but assignment fails, preserve the distinction and resolve through an authorized demo workflow.

If offline delivery cannot be verified, state that the payload remains unconfirmed. Do not clear browser storage before preserving queued reports. If role changes cause confusion, use separate profiles and verify the displayed account before each action.

If native Android footage is unavailable, show the responsive web view and describe the existing Capacitor packaging path. Do not label a browser frame as a tested native APK or claim an app-store release. If source behavior changes, update both this script and the complete guide before recording.

21. Suggested interview and evaluator questions

Question: Why combine AI with deterministic rules? Suggested answer: “The local engine provides a transparent baseline without a cloud key. A configured model can add another interpretation, while validation and human review remain necessary. We do not claim the ensemble is clinically calibrated.”

Question: What is explainable here? Suggested answer: “Reports retain their text, triage includes reason codes, priority exposes six numeric factors, and recommendations include suitability/distance context. Some audit events record decisions, but the audit trail is not yet comprehensive or tamper-proof.”

Question: What prevents automatic dispatch? Suggested answer: “The dashboard requires a dispatcher/admin approval action, and assignment creation is role-restricted. The optimizer only returns a plan. Authorized manual assignments are also supported through the API.”

Question: What would be needed before real use? Suggested answer: “A separate safety and operational program: validated requirements and data, expert evaluation, privacy/legal review, secure account policy, reliable communications, complete workflow integrity, tested recovery, and independent assurance. This prototype must not be used for real emergencies.”

Question: What makes this more than a map? Suggested answer: “The map is only one view. The project connects ownership-controlled reports, structured triage, a score, resource proposals, human approval, assignments, and field progress. The value being demonstrated is the connected workflow.”

22. Ninety-second cut-down script

Use this only as a separate short trailer or presentation introduction. Keep the complete film and guide available for the detailed explanation. Read naturally; trim pauses during rehearsal rather than accelerating speech excessively.

Narration:

“ResQGrid AI is a demonstration of an intelligent emergency resource network. It brings fictional incident reports, map context, advisory triage, and human-approved assignments into one web interface.

A citizen submits a report and checks its location. A dispatcher reviews the original information, runs local and optionally cloud-assisted triage, then calculates an explainable six-factor priority score. The system suggests available resources using type, approximate distance, range, and hazard context. Travel times are estimates, not live routing guarantees.

A person decides whether to approve. When assignment succeeds, a responder can accept the task, mark progress, and complete it. The resource returns to the available pool, while incident closure remains separate.

The project also includes protected image evidence, a database-grounded command assistant, partial offline report queuing, and an API-based global assignment optimizer. It uses Next.js, FastAPI, PostgreSQL, and optional cloud integrations, with an Android packaging path through Capacitor.

This is not certified emergency software. Its data must remain fictional, its confidence scores are not validated outcomes, and its public demo accounts are intentionally preserved. The principle is simple: AI assists. Humans approve.”

Visual sequence: Title/safety card → report form → triage/score → candidate/approval → responder lifecycle → architecture → limitations/closing card. Do not include unsupported generated footage or performance claims.

23. Editorial quality checklist and sources

Before delivery, verify that the narration matches the actual recorded revision; all scenario data is fictional; the active AI mode is labeled accurately; no secrets, personal data, or private browser content appear; map/media attribution remains visible; captions preserve technical terms; score examples are identified as examples; the optimizer is labeled API-only; and future work is clearly separated from implemented functionality.

Confirm the film does not imply guaranteed real-time updates, official dispatch, validated safest routing, automatic incident closure, reliable closed-app background delivery, comprehensive immutable auditing, unrestricted AI knowledge, or an open-source license. Verify the final duration against the edited video and update the title card if it differs materially from the target.

This script derives from the source-based complete guide, `docs/PROJECT_DOCUMENTATION.md`, at baseline 0d85c00. Key implementation sources are `Dashboard/DetailPanel/ReportIncidentModal`, the API routes, local triage/ensemble, priority/recommendation services, assignment optimizer, auth/object-access policy, evidence/vision/assistant services, and browser outbox/service worker. The project's LICENSE and DISCLAIMER govern legal and safety statements.

Generate the editable Word and presentation-ready PDF editions using `python docs/generate_documents.py`. The output files are `docs/exports/ResQGrid-AI-Documentary-Script.docx` and `.pdf`. This package provides the script and production instructions; it does not claim that voice-over, screen recording, editing, subtitles, or a finished video have already been produced.